

Experiment - 2

Develop a mapreduce program to implement Matrix Multiplication

Remove the existing eclipse IDE in the system.

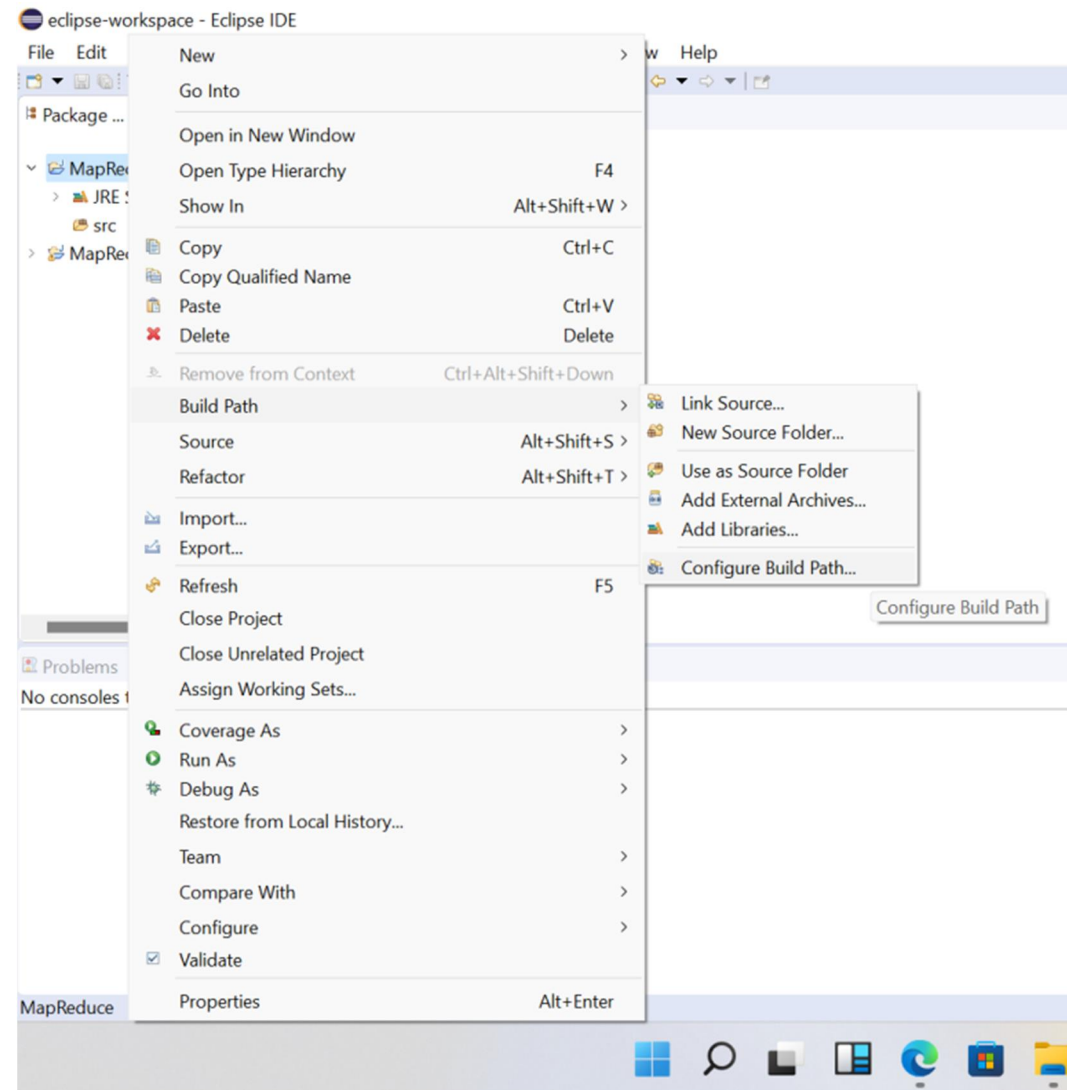
Refer a below link to download and install Eclipse version: Eclipse IDE 2019-12

<https://www.eclipse.org/downloads/packages/release/2019-12/r/eclipse-ide-java-developers>

Step 1: Open Eclipse, create a Java project by any name, create a package and give a name for it.

Add external jars which are required for the programs by following the steps given below:

- 1) Right click on the project that have been created, choose an option called as “build path”, then choose “configuration build path”.
- 2) choose “Java Build Path”, then “Libraries” and then “Add External JAR’s”. Refer to the image in the next slide.



Properties for MapReduce

- > Resource
- Builders
- Coverage
- Java Build Path
- > Java Code Style
- > Java Compiler
- > Java Editor
- Javadoc Location
- Project Natures
- Project Reference:
- Run/Debug Settir
- > Task Repository
- Task Tags
- > Validation
- WikiText

Java Build Path

Source Projects Libraries Order and Export Module Dependencies

JARs and class folders on the build path:

> JRE System Library [JavaSE-1.8]

Add JARs...

Add External JARs...

Add Variable...

Add Library...

Add Class Folder...

Add External Class Folder...

Edit...

Remove

Migrate JAR File...

Apply



Apply and Close

Cancel

3) After choosing an option “Add External JAR’s”, select all the JAR files in the path **C:\hadoop\share\hadoop**

Select each folder of it, and choose all the JAR files and add it to the JRE System library.

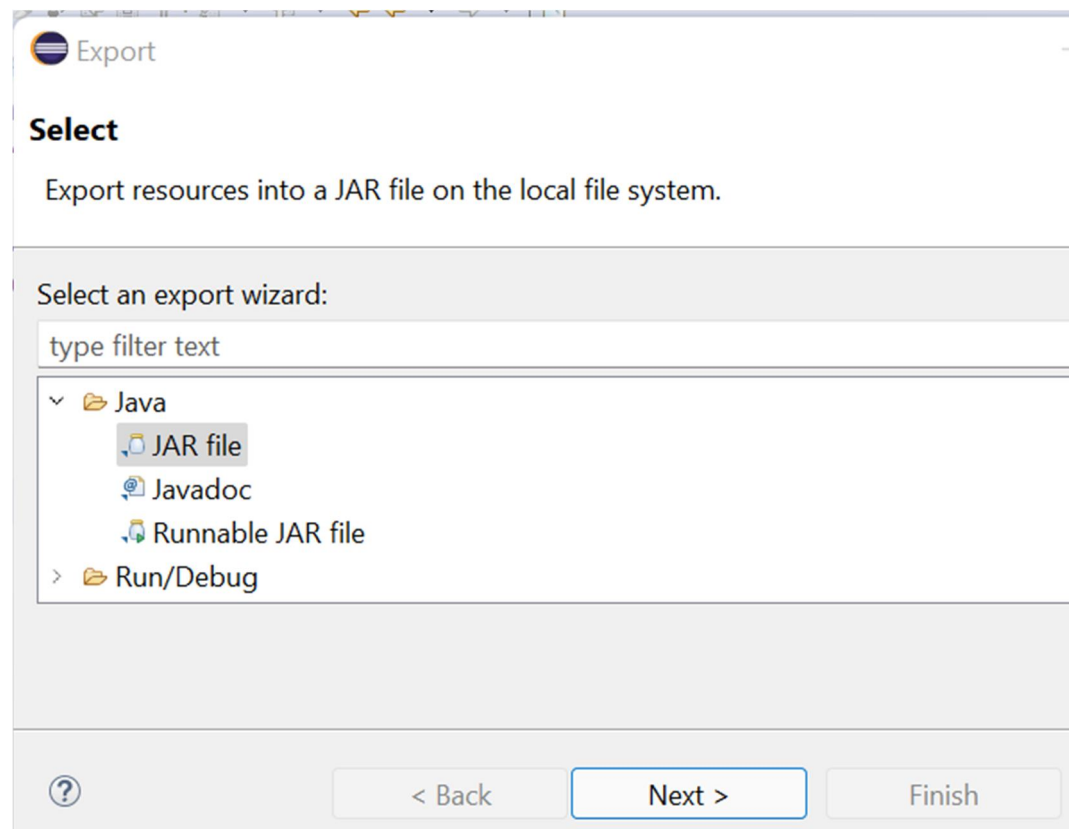
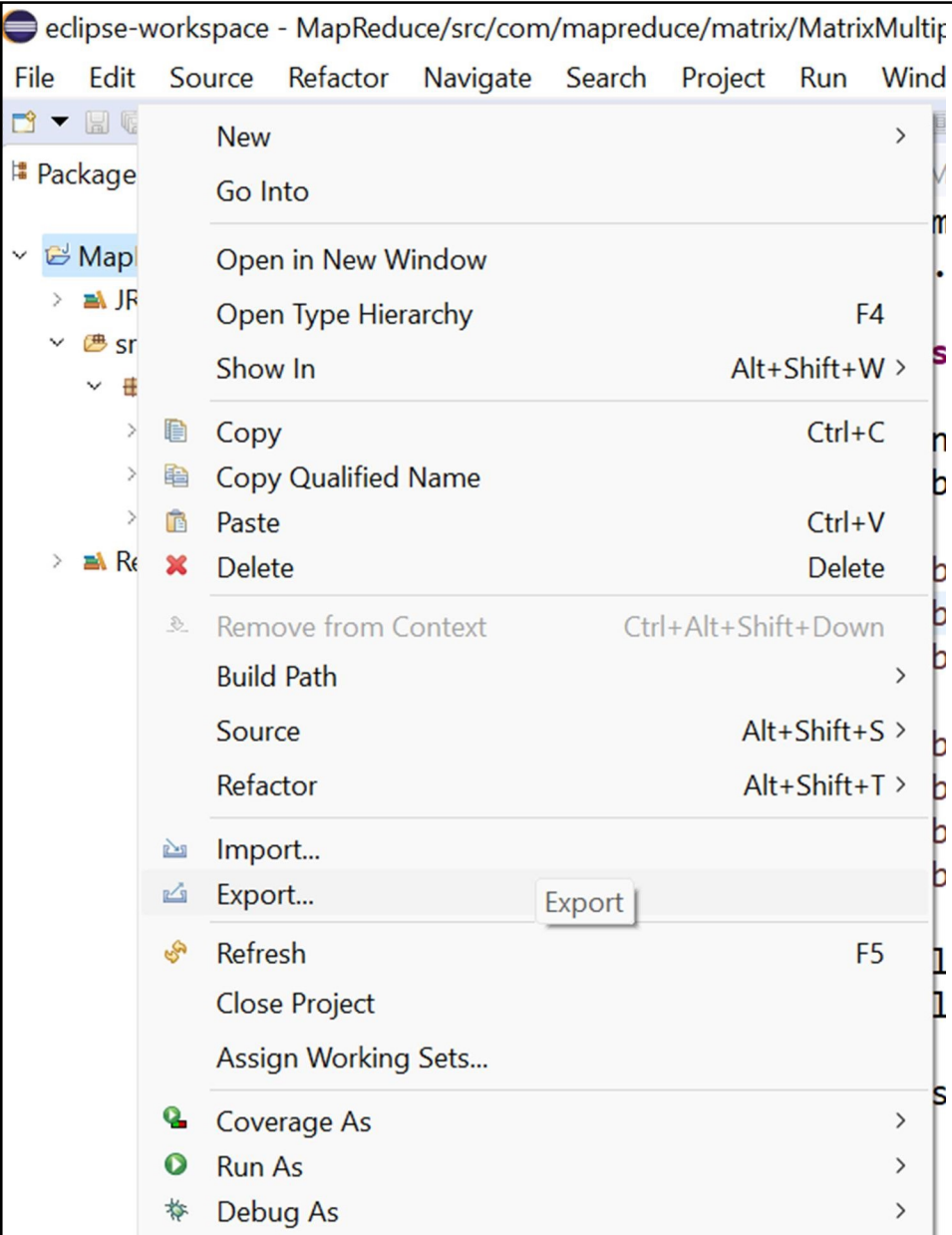
4) Create 3 class files, one for Mapper, Reducer and Driver.

In our program, we are going to create files named:

- MatrixMultiplication.java
- MatrixMultiplicationMapper.java
- MatrixMultiplicationReducer.java

Add the code in all these files, and press Ctrl+S to save it.

5) Create a jar file of your project, by right clicking on created java project, then choose “export”, in that, choose “Java”, and then choose “JAR file” option given, then click on “Next”. Give a JAR file name that you want to create and give a path where it need to be saved. Refer the screenshots given in the next slide.



JAR File Specification

Define which resources should be exported into the JAR.

Select the resources to export:

>  MapReduce	☒  .classpath ☒  .project
--	--

- ☒ Export generated class files and resources
- ☐ Export all output folders for checked projects
- ☐ Export Java source files and resources
- ☐ Export refactorings for checked projects. [Select refactorings...](#)

Select the export destination:

JAR file: 

Options:

- ☒ Compress the contents of the JAR file
- ☐ Add directory entries
- ☐ Overwrite existing files without warning



After creating the JAR file of the project, follow the below given commands.

Step 2: Open **command prompt** in **administrator mode**.

Step 3: start all the daemons of hadoop by giving a command: **start-all.cmd**

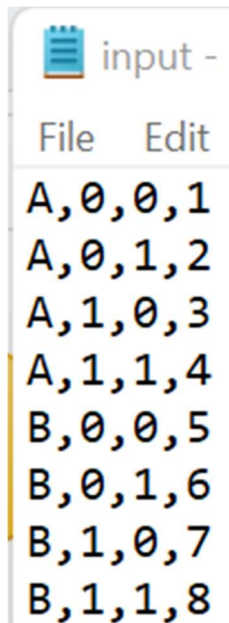
```
C:\Windows\system32>cd \  
  
C:\>cd hadoop  
  
C:\hadoop>cd sbin  
  
C:\hadoop\sbin>start-all.cmd
```

Step 4: create a directory in hdfs by using a command:

hadoop fs -mkdir /directoryname

```
C:\hadoop\sbin>hadoop fs -mkdir /input_matrix
```

Step 5: create a text file in any of the directory in your computer. Store the inputs to be given for the program. Text file name should be : **input.txt**



input -

File	Edit
A,0,0,1	
A,0,1,2	
A,1,0,3	
A,1,1,4	
B,0,0,5	
B,0,1,6	
B,1,0,7	
B,1,1,8	

Step 6: add the text file to the directory that is created in HDFS previously by using the given command: `C:\hadoop\sbin>hadoop fs -put E:\input.txt /input_matrix`

To verify whether the file exists or not in the directory of hdfs, check in the localhost:9870

Or

By typing a command: `C:\hadoop\sbin>hadoop fs -ls /input_matrix`

To display the contents of the text file in the command prompt, use a command as given below:

```
C:\hadoop\sbin>hadoop dfs -cat /input_matrix/input.txt
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

Step 7: To execute the program, follow the below given command:

```
C:\hadoop\sbin>hadoop jar E:\Matrixmul.jar com.mapreduce.matrix/MatrixMultiplication /input_matrix/input.txt /output_matrix
```

Path of projects JAR file

Path(path from the package of current working java project) of the java file which consists of main method.

It's a HDFS path where the input.txt file is present, which is the input of the program.

It's a path where the output of the program will be stored in the form of file.

```
2025-03-15 22:32:55,810 INFO mapreduce.Job: map 0% reduce 0%
2025-03-15 22:33:02,986 INFO mapreduce.Job: map 100% reduce 0%
2025-03-15 22:33:11,096 INFO mapreduce.Job: map 100% reduce 100%
2025-03-15 22:33:11,106 INFO mapreduce.Job: Job job_1742057507536_0001 completed successfully
```

Step 8: To check the output, type a command as given below:

```
C:\hadoop\sbin>hadoop dfs -cat /output_matrix/*
```

```
0,0      19  
0,1      22  
1,0      43  
1,1      50
```

output

```
package com.mapreduce.matrix;

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class MatrixMultiplicationMapper extends Mapper<LongWritable, Text, Text, Text> {
    private static final int MATRIX_SIZE = 2; // Set to correct matrix dimensions (2x2 in your case)

    public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
        String[] tokens = value.toString().split(",");
        String matrixName = tokens[0]; // "A" or "B"
        int i = Integer.parseInt(tokens[1]);
        int j = Integer.parseInt(tokens[2]);
        int val = Integer.parseInt(tokens[3]);

        if (matrixName.equals("A")) {
            for (int k = 0; k < MATRIX_SIZE; k++) { // Iterate correctly
                context.write(new Text(i + "," + k), new Text("A," + j + "," + val));
            }
        } else if (matrixName.equals("B")) {
            for (int k = 0; k < MATRIX_SIZE; k++) { // Iterate correctly
                context.write(new Text(k + "," + j), new Text("B," + i + "," + val));
            }
        }
    }
}
```

```
import org.apache.hadoop.io.*;
```

- imports all classes inside the org.apache.hadoop.io package. These classes handle **Hadoop-specific data types** that are optimized for distributed computing.

```
import org.apache.hadoop.mapreduce.Mapper;
```

- This imports the **Mapper** class, which is a part of the Hadoop MapReduce framework.

```
import java.io.IOException;
```

- This imports the **IOException** class from Java's built-in **java.io** package.

```
package com.mapreduce.matrix;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.HashMap;

public class MatrixMultiplicationReducer extends Reducer<Text, Text, Text, IntWritable> {
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
        HashMap<Integer, Integer> aValues = new HashMap<>();
        HashMap<Integer, Integer> bValues = new HashMap<>();

        for (Text val : values) {
            String[] parts = val.toString().split(",");
            if (parts[0].equals("A")) {
                aValues.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
            } else {
                bValues.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
            }
        }

        int sum = 0;
        for (int k : aValues.keySet()) {
            if (bValues.containsKey(k)) {
                sum += aValues.get(k) * bValues.get(k);
            }
        }

        context.write(key, new IntWritable(sum));
    }
}
```

`org.apache.hadoop.io.*`

Brings in Hadoop's optimized data types (`Text` , `IntWritable`).

`org.apache.hadoop.mapreduce.Reducer`

Imports the Hadoop **Reducer** class to define the reduce function.

`java.io.IOException`

Handles **input-output exceptions** during file processing.

`java.util.HashMap`

Provides a data structure to store **intermediate values** (A and B matrix elements).

```
package com.mapreduce.matrix;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MatrixMultiplication {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Matrix Multiplication");

        job.setJarByClass(MatrixMultiplication.class);
        job.setMapperClass(MatrixMultiplicationMapper.class);
        job.setReducerClass(MatrixMultiplicationReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```